

IAE Project Report

Karthik Sundram
Roll No: 2024101130

Abdul Ahad Shoeb
Roll No: 2024101136

1 Introduction

Evaluating the diameter and eccentricity of large-scale graphs is a foundational problem in network analysis. Computing these metrics exactly typically requires an All-Pairs Shortest Path (APSP) computation, which is prohibitively expensive for massive real-world graphs. Parallelization of multiple searches (mainly through BFS) is one effective way to tackle this. In this report, we will be comparing the k-BFS algorithm and the RV approximation algorithm against their performance for different types of graphs.

2 Methodology

2.1 k-BFS Algorithm

The k-BFS algorithm is an efficient, two-phase heuristic approach used to estimate the eccentricity of vertices in a graph.

- **Method:** In the first phase, the algorithm selects a random sample of k source vertices and runs Breadth-First Searches (BFS) from them to identify the k vertices that are furthest away (often landing on the graph's periphery). In the second phase, it runs another set of BFSs from these k newly discovered furthest vertices to compute the final eccentricity estimates.
- **Number of BFS Operations:** Exactly $2k$ total searches (executed as k parallel searches per phase).
- **Time/Work Complexity:** $\mathcal{O}(km)$, where m is the total number of edges in the graph.

2.2 RV Algorithm

The RV algorithm (developed by Roditty and Vassilevska Williams) is an approximation algorithm that provides a strict theoretical bound on the eccentricity estimates.

- **Method:** It operates by selecting a random sample of vertices S proportional to $\sqrt{n \log n}$. It executes BFSs from this sample, identifies the vertex w that is furthest from any sampled node, and then explores the immediate neighborhood $N_s(w)$. The algorithm computes exact eccentricities for these local sets and uses them to bound the estimates for the rest of the graph.
- **Number of BFS Operations:** $\Theta(\sqrt{n \log n})$ searches, driven largely by the initial sample size and the size of the farthest vertex's neighborhood.
- **Time/Work Complexity:** $\mathcal{O}(m\sqrt{n \log n})$, where n is the number of vertices and m is the number of edges.

3 Results

Table 1 summarizes the experimental evaluation of the k-BFS and RV algorithms across multiple real-world graph datasets. The results highlight the estimated diameter, the total number of BFS operations performed, and the running time for various configurations.

Table 1: Performance Evaluation of k-BFS and RV Algorithms

Graph Dataset	Nodes / Edges	Ground Truth	Algorithm	Est. Diameter	BFS Count	Time (ms)
CA-CondMat	21,363 / 91,286	14	k-BFS (k=1)	15	2	1.635
			k-BFS (k=2)	15	4	2.674
			k-BFS (k=4)	15	8	3.944
			k-BFS (k=8)	15	16	5.105
			k-BFS (k=16)	15	32	5.928
			k-BFS (k=32)	15	64	7.964
			k-BFS (k=64)	15	128	9.104
			RV ($\sqrt{n}$)	15	924	468.028
CA-HepTh	8,638 / 24,806	17	k-BFS (k=1)	18	2	0.571
			k-BFS (k=2)	18	4	0.972
			k-BFS (k=4)	18	8	1.467
			k-BFS (k=8)	18	16	1.975
			k-BFS (k=16)	18	32	2.176
			k-BFS (k=32)	18	64	2.795
			k-BFS (k=64)	18	128	3.459
			RV ($\sqrt{n}$)	17	561	100.483
Email-Enron	33,696 / 180,811	11	k-BFS (k=1)	13	2	2.374
			k-BFS (k=2)	13	4	3.156
			k-BFS (k=4)	13	8	5.319
			k-BFS (k=8)	13	16	6.498
			k-BFS (k=16)	13	32	10.262
			k-BFS (k=32)	13	64	10.643
			k-BFS (k=64)	13	128	11.482
			RV ($\sqrt{n}$)	13	1187	748.063
Wiki-Vote	7,066 / 100,736	7	k-BFS (k=1)	7	2	0.909
			k-BFS (k=2)	7	4	1.356
			k-BFS (k=4)	7	8	1.966
			k-BFS (k=8)	7	16	2.593
			k-BFS (k=16)	7	32	3.137
			k-BFS (k=32)	7	64	3.592
			k-BFS (k=64)	7	128	4.117
			RV ($\sqrt{n}$)	7	502	99.665
email-Eu-core	986 / 16,064	7	k-BFS (k=1)	6	2	0.158
			k-BFS (k=2)	6	4	0.188
			k-BFS (k=4)	7	8	0.295
			k-BFS (k=8)	7	16	0.477
			k-BFS (k=16)	7	32	0.638
			k-BFS (k=32)	7	64	0.723
			k-BFS (k=64)	7	128	0.801
			RV ($\sqrt{n}$)	7	166	4.888
facebook_combined	4,039 / 88,234	8	k-BFS (k=1)	8	2	0.333
			k-BFS (k=2)	8	4	0.369
			k-BFS (k=4)	8	8	0.484
			k-BFS (k=8)	8	16	0.668
			k-BFS (k=16)	8	32	0.803
			k-BFS (k=32)	8	64	1.064
			k-BFS (k=64)	8	128	1.637
			RV ($\sqrt{n}$)	8	368	39.597

4 Conclusion

Based on the experimental data from the six graph datasets, we draw the following key conclusions:

- **Efficiency vs. Cost:** The k-BFS algorithm is orders of magnitude faster than the RV algorithm. For example, on the Email-Enron graph, the most intensive k-BFS run ($k = 64$) took roughly 11.5 ms, whereas the RV algorithm required 748 ms. The RV algorithm’s strict theoretical bounds come at a heavy computational cost due to its $\Theta(\sqrt{n \log n})$ BFS operations.
- **Rapid Convergence:** The k-BFS method requires very small k values to stabilize. Across all tested datasets, the diameter estimate converged almost immediately at $k = 1$ or $k = 2$. Increasing k further only added computation time without improving the estimate.
- **Accuracy Edge of RV:** While k-BFS is exceptionally fast, it occasionally overestimates the diameter slightly (e.g., CA-CondMat). The RV algorithm more consistently matched the exact ground truth.

Final Verdict: For practical, large-scale applications where speed is critical, k-BFS with a low k value is vastly superior. However, for rigorous academic or analytical contexts that strictly require mathematical bounds and exact ground truth accuracy, the RV algorithm remains necessary despite its performance penalty.